Optimizing the Inference Pipeline of a Fixed Small VLM: An AlphaEvolve-Style Study on CharXiv

Tejas Naladala
naladala@uw.edu

Abstract

We study how far the inference pipeline around a fixed small vision–language model (Qwen3-VL-2B) can be pushed on the CharXiv chart-understanding benchmark. The grader is a case-insensitive exact-match test, and decoding must be greedy. We do three things. First, we build an AlphaEvolve-style evolutionary system: a local LLM mutates an inference program against a fitness signal, with a MAP-Elites archive over accuracy and latency and a sandbox for candidate code. Second, we hand-tune the same `vlm_inference` function. Third, we compare evolved and manual pipelines on the Instruct and Thinking model variants. All headline numbers use a three-way disjoint split: an official 128-item scoreboard, a development slice for all tuning and ablation, and a held-out slice scored once. A naive baseline scores 29.5% held-out. Manual prompting and answer extraction reach 60.5%. The evolutionary search finds a faster single-prompt configuration at 63.25% and 0.16 s/query. A hand-built per-question-type router reaches 73.0% held-out at sub-second latency. That is 2.5 times the naive held-out baseline, and 2.0 times on the official 128-item slice. We give Wilson 95% confidence intervals for every accuracy and significance tests for every headline comparison. A paired McNemar test on the held-out 400 shows the router beats the single-prompt configuration item by item (b=52 vs. c=13, p=2.4e$-$6). We also report negative results. Crop-zoom lowered accuracy. The Thinking model is 20 to 40 times slower for lower exact-match accuracy. We state where each number was measured, including one Thinking number that is in-sample and is reported as directional.

1 Introduction

CharXiv [1] asks descriptive questions about real scientific charts. The provided evaluator marks a response correct only on a case-insensitive exact string match with the ground truth, `response.strip().lower() == gt.lower()`. The task also requires greedy decoding, `do_sample=False`, for reproducibility. These two rules change the problem. Most of the gap for a naive baseline is not about perception. It is about formatting. The model often reads the chart correctly but answers in a full sentence, so its output never matches the bare canonical string. This makes the inference pipeline the main lever: the prompt, the answer extraction, the image resolution, the question-type routing, and any deterministic ensembling. That makes it a good target for an automated optimizer.

Following AlphaEvolve [2], we build an evolutionary system. An LLM proposes edits to an inference program. An automated evaluator scores each candidate on real chart questions. A quality-diversity archive keeps an accuracy–latency frontier. We compare the evolved programs against our own manual tuning, on two 2B model variants.

Preprint. Under review.

2 Problem setup

Models. Qwen3-VL-2B-Instruct and Qwen3-VL-2B-Thinking. Both are fixed; we do not swap in a larger model. We run them in bf16 on a single H100 and decode greedily throughout (`do_sample=False`, `num_beams=1`).

Benchmark and grader. We use the CharXiv descriptive split. The grader is `response.strip().lower() == gt.lower()`. Answers are short canonical strings: numbers, axis labels, `Yes` or `No`, layouts like `1 by 2`, and enumerated legend labels. About 24% of all answers are `Not Applicable`, because the chart often lacks the queried element. The question templates already state the exact answer format and the rule to answer `Not Applicable` when the element is absent.

Evaluation protocol. The `descriptive_val` split has 1000 graded questions. We split them into three disjoint slices so that no number we tune on is also a number we report as a generalization estimate.

- **Official** = `items[:128]`. The 128-item scoreboard that `reproduce.sh` reproduces.
- **Dev** = `items[400:600]` (200 items). Used for all `best_accuracy` tuning and the ablation in Appendix A.
- **Held-out** = `items[600:1000]` (400 items). Scored once per configuration. This is the headline number.

There are two tuning surfaces, and the held-out slice is disjoint from both. The evolutionary search selects its single-prompt program on `items[:96]`, which is the subset used for the fitness curve in Figure 1. The structural levers were tuned on `items[400:600]`. These two surfaces do not overlap, and `items[600:1000]` overlaps neither. The official 128-item scoreboard does contain the evolution selection set `items[:96]` as a prefix, so the evolved Instruct accuracy on the official slice is partly in-sample, like the two Thinking numbers; we therefore treat the disjoint held-out 400 as the clean evolved estimate and read the official evolved number as directional.

Naive baseline. The provided `starting_scripts.py` returns the model's full prose answer with a large token budget. It scores 38.3% on the official slice (0.92 s/query) and 29.5% on the held-out 400 (0.97 s/query).[1] We updated its deprecated `AutoModelForVision2Seq` import to `AutoModelForImageTextToText` for transformers 5.x.

3 Method

3.1 Manual optimization

The grader rewards only the bare canonical answer. Our manual `vlm_inference` does three things. It uses a short system prompt that forbids explanation and asks for the literal printed value, keeping the sign, decimals, and exponent notation. It tells the model to answer exactly `Not Applicable` when the queried element is absent, which is the largest answer class. And it post-processes the output through a small extraction layer (`charxiv_answer.py`, 28/28 unit tests) that strips prose wrappers, canonicalizes the `Not Applicable`, `Yes`, `No`, and layout forms, normalizes spacing in enumerated labels, and leaves numbers untouched. With a small `max_new_tokens`, this reaches 62.5% on the official slice and 60.5% held-out. That is a 24 to 31 point gain over naive, and it is 4 to 6 times faster, from the prompt and extraction alone.

3.2 The evolutionary system

We implement the core of AlphaEvolve from scratch, without a third-party evolution framework.

- **Genome.** A candidate is a small sandboxed Python snippet. It defines `SYSTEM`, `USER_SUFFIX`, `GEN_KWARGS` (forced greedy), `IMAGE` (resolution), and a `postprocess(raw, ques-`

[1] Per-query latency varies a few percent across cluster nodes; an earlier single-node run of the same naive configuration logged 0.85 s/query in `cluster/logs/baseline_naive.json`. The accuracy is identical; we quote the run used for Table 1.

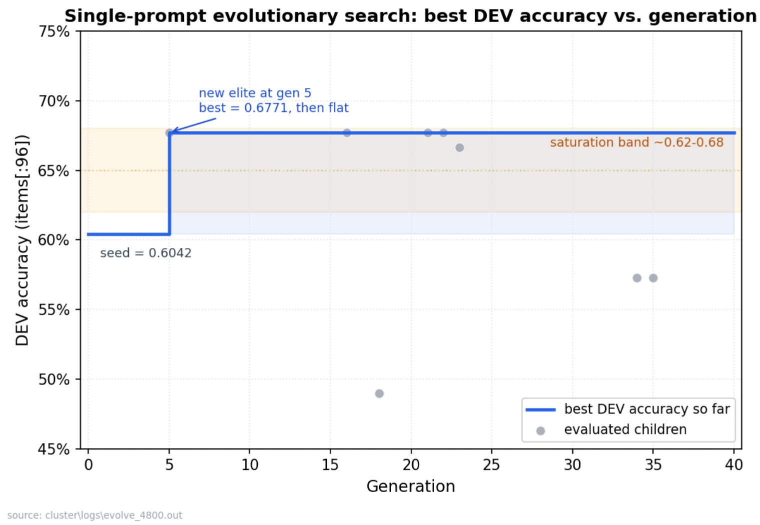

Figure 1: Single-prompt evolutionary search over 40 generations on the `items[:96]` selection subset. The best dev accuracy (solid line) rises from the 0.6042 seed to 0.6771 at generation 5 and then stays flat. Evaluated children (points) scatter from about 0.49 to 0.68, below the best-so-far line. The flat curve is why further accuracy has to come from structural ensembling rather than more search.

`tion)` function. The model is loaded once and reused, so a candidate evaluation is just forward passes.

- **Variation.** A child comes from one of two operators. Mutation has a local Qwen2.5-Coder rewrite one parent, given its score, per-type accuracy, and concrete mistakes. Crossover splices the genes {`SYSTEM`, `USER_SUFFIX`, `GEN_KWARGS`, `IMAGE`, `postprocess`} from two parents through the same AST-safe splicer. `CROSSOVER_RATE` sets the mix. Either way the child stays sandbox-valid and greedy. Appendix B ablates crossover against mutation only.
- **Archive.** MAP-Elites over accuracy×latency cells, with island-style parent sampling. This keeps a diverse frontier.
- **Evaluator and safety.** A cascade probe drops weak candidates early. Every candidate runs in an AST sandbox under a per-candidate wall-clock timeout. The sandbox pins `__builtins__` to a small whitelist, so aliased or `chr(95)`-reconstructed `__import__`-/`open`/`getattr` do not exist at runtime. Imports are limited to `re`, `math`, and `json`, and `GEN_KWARGS` and resolution are clamped. Greedy decoding is re-enforced in the written artifact, so an evolved program cannot ship a non-greedy setting.
- **Anti-overfit.** Candidates are selected on dev. The best of the dev top-k is chosen on a disjoint validation split. The single winner is scored on a held-out slice once.

The single-prompt search saturates quickly (Figure 1). On the `items[:96]` selection subset, the best dev accuracy rises from the 0.6042 seed to 0.6771 at generation 5, then stays flat for the remaining 35 generations. This is the negative finding that motivates the structural router in Section 3.3. A fixed-shape genome cannot express the per-type routing, gating, and multi-pass voting that break the single-prompt ceiling.

3.3 Breaking the single-prompt ceiling (`best_accuracy`)

A single prompt saturates near 63 to 65% here. For `best_accuracy` we break it with steps that use several deterministic passes. Decoding stays greedy, so the result is still reproducible. The configuration is Qwen3-VL-2B-Instruct plus four parts. First, a per-question-type keyword router sends tick and label questions to a high-resolution OCR prompt with exponent and Greek normalization. Second, existence gates run a yes/no probe ("is there a colorbar, line, title, or legend?") and return `Not Applicable` when the element is absent, before the model can invent a value. Third, closed-class constrained decoding masks the first token for Yes/No and <int> by <int> layout answers.

Table 1: Main results (Instruct). Intervals are Wilson 95%. `best_speed` is `evolved_instruct`. `best_overall` and `best_accuracy` share the router and differ only in whether the vote is on.

Configuration	Model	Official-128 [95% CI]	Held-out-400 [95% CI]	s/q (off / held)
naive `starting_scripts`	Instruct	38.3 [30.3, 46.9]	29.5 [25.2, 34.1]	0.92 / 0.97
`manual_instruct`	Instruct	62.5 [53.9, 70.4]	60.5 [55.6, 65.2]	0.22 / 0.155
`evolved_instruct` (=`best_speed`)	Instruct	64.8 [56.2, 72.6]	63.25 [58.4, 67.8]	0.23 / 0.162
`best_overall` (router, no vote)	Instruct	77.3 [69.4, 83.7]	73.0 [68.4, 77.1]	0.379 / 0.430
`best_accuracy` (router + vote)	Instruct	**77.3** [69.4, 83.7]	**73.0** [68.4, 77.1]	0.403 / 0.452

This is logit masking only; the greedy argmax is unchanged. Fourth, the guess-prone tick-step template uses multi-resolution greedy voting. All passes are single-model, greedy, and deterministic. A pure-Python self-test for routing, normalization, constrained-id computation, and flag parsing passes 87/87 on CPU without the model.

4 Results

All accuracies are exact-match. All latencies are seconds per query, measured on the same run as the accuracy in that row. Instruct rows are scored on the official 128 and the held-out 400. The Wilson 95% confidence intervals and all significance tests are computed by `cluster/stats.py`, with output in `cluster/STATS.md`, from the logged `(correct, n)` counts. The held-out latency is the convention used throughout, with the official latency given for reference.

Significance. Our main test is the paired McNemar test, because `best_accuracy` and `evolved_instruct` are scored on the same held-out 400. They agree on 335 items and disagree on 65. The split is b=52 (`best_accuracy` right, evolved wrong) versus c=13 (evolved right, `best_accuracy` wrong). The continuity-corrected statistic is χ^2=22.2, p=2.4e−6 (log `cluster/logs/mcnemar_4833.out`). So the router does not just win on aggregate accuracy. It wins item by item, fixing about four wrong-to-right answers for each one it breaks. The unpaired two-proportion z-tests agree. `best_accuracy` 73.0% versus naive 29.5% gives z=12.31, $p\approx$8e−35. `best_accuracy` 73.0% versus `evolved_instruct` 63.25% gives z=2.96, p=0.003. By contrast, `evolved_instruct` 63.25% versus `manual_instruct` 60.5% gives z=0.80, p=0.42, which is not significant. The per-knob dev ablation deltas, from +0.5 to +2.0 points on 200 items, all sit inside the ±6.6-point Wilson half-width, so we treat them as directional steps. The two significant accuracy gains are the full pipeline over the naive baseline and over the single-prompt evolved configuration.

Headline, and which denominator. The router reaches 73.0% held-out, which is 2.5 times the naive held-out baseline of 29.5%. On the official 128-item slice the same comparison is $77.3/38.3 = 2.0$ times. Both are within-slice comparisons. The held-out 400 is the larger and fully disjoint slice, so we treat the held-out 2.5 times as the main number and state the official 2.0 times next to it.

Three operating points. The brief asks for three configurations. We ship three that span the frontier (Figure 2). Two of them share held-out accuracy, and we state that directly rather than add a middle number that is not real.

- **Best speed** is `evolved_instruct`: one greedy single-prompt call, no gates or routing. This is the distinct low-latency point, about 0.16 s/query held-out at 63.25%.
- **Best overall** is the router without the multi-resolution vote, so one high-resolution `ocr_step` read instead of two. It is 73.0% held-out at about 0.43 s/query (log `cluster/logs/heldout_4836.out`).
- **Best accuracy** is the same router with the vote kept. It is 73.0% held-out at about 0.45 s/query. It keeps the second voting pass for robustness on the tick-step template.

A note on `best_overall` versus `best_accuracy`. For this grader and this fixed 2B model, the router with the vote and the router without it reach the same held-out accuracy, 73.0%. The ablation in Appendix A found the vote accuracy-neutral on these slices. So the two are distinct latency points at equal accuracy, not distinct accuracy points. `best_speed` is the one point that trades real accuracy for real speed.

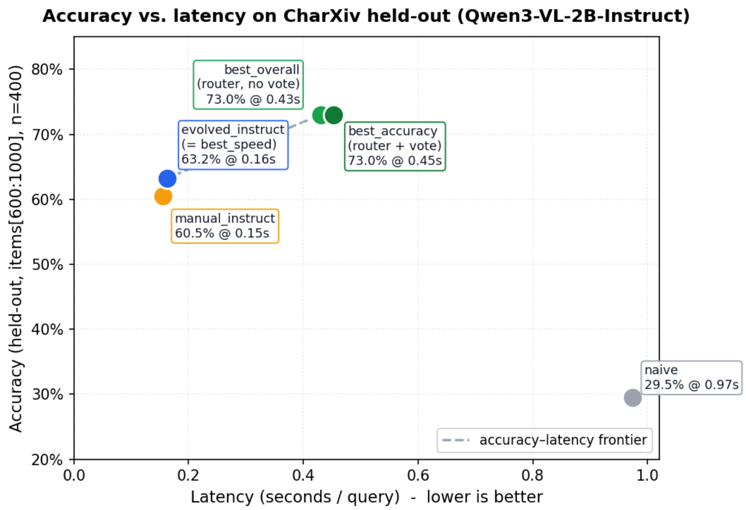

Figure 2: Accuracy on the held-out 400 versus latency, for the five configurations. Both router variants reach 73.0%, with `best_overall` at slightly lower latency. The router leads on accuracy at sub-second latency. `evolved_instruct` (63.25% at 0.16 s/q) and `manual_instruct` (60.5%) sit at the fast end. The naive baseline is dominated by every other point and is not on the frontier line.

4.1 Evolution versus manual

On both slices the evolved Instruct program edges the hand-tuned baseline on accuracy: 64.8 versus 62.5 on the official slice, and 63.25 versus 60.5 held-out. Speed is comparable. As the significance test shows, the held-out edge of +2.75 points (p=0.42) is within noise. The headline 73.0% does not come from the evolutionary search. It comes from the hand-built router in Section 3.3. The single-prompt genome saturates near 63 to 65%. So evolution contributes in two ways. It produced the best single-prompt configuration, `best_speed`, plus a frontier a human did not hand-find. And it produced a clear saturation result, the single-prompt ceiling, which motivated the router that reaches 73.0%. A quality-diversity ablation in Appendix B shows the LLM-mutator with MAP-Elites beats a random-restart baseline on both the dev objective and a once-scored held-out winner.

4.2 Instruct versus Thinking

The Thinking model is 20 to 40 times slower than Instruct, and its exact-match accuracy is lower than `best_accuracy`. For a grader whose answers are a few characters, the long chain of thought is a net loss. Evolution still improved Thinking on its own terms: `manual_thinking` 57.3% rose to `evolved_thinking` 62.5%, and it got faster, from 8.67 to 6.30 s/query, by tightening the final answer scaffold and trimming the reasoning budget.

One caveat about measurement. The Instruct held-out numbers use `items[600:1000]`, which is disjoint from all tuning. The two Thinking numbers were measured on `items[:96]`, which is also the Thinking arm's selection set, because of the model's high per-query cost. The +5.2 point evolved-over-manual Thinking gain is therefore in-sample. We report it as directional, not as a clean held-out gain, and Thinking does not carry any headline.

5 Analysis

- The biggest accuracy lever is making the model emit only the canonical answer, plus robust extraction. This alone is the 24 to 31 point gain over naive, because the gap is formatting, not perception.
- `Not Applicable` calibration through existence gates is the highest-value structural lever, since about 24% of answers are `Not Applicable`. The legend gate alone is +2.0 on dev (Appendix A).

- Evolution added a faster, slightly more accurate single-prompt configuration and a diverse frontier. But the search saturates, and breaking the ceiling needed hand-designed structural ensembling.
- Negative results. Crop-zoom of the axis strips lowered dev accuracy by 0.5 points and was turned off. Thinking is a net loss under exact match. Lower resolution barely improves latency, because the cost is fixed overhead at these sizes, and it lowers accuracy. Aggressive comma and period normalization can corrupt valid answers, so we restricted it. Appendix A lists the full set of dead ends: batching, the axis gate, an OCR-resolution sweep, and a fresh 40-generation rerun.

The gap from naive to 73.0% is formatting under exact match, not perception. The naive model often reads the right value but answers in prose, for example "The maximum value shown is approximately 0.8", which never matches the bare `0.8`. Or it invents a value for an element that is absent, instead of answering `Not Applicable`. Each lever targets one such formatting or calibration failure.

6 Methodology and limitations

Reporting. Earlier versions of `best_accuracy` were developed by inspecting failures on a 128-item slice. We do not report any in-sample peak on that slice. All headline numbers use the disjoint protocol in Section 2: dev (`items[400:600]`) for all tuning and ablation, and held-out (`items[600:1000]`) scored once. Two numbers are measured on a slice that contains their own selection set and are reported as directional: the Thinking gain (Section 4.2) and the evolved Instruct accuracy on the official 128, whose first 96 items are the evolution selection set. Both held-out 400 numbers, the evolved 63.25% and the router 73.0%, are disjoint from every tuning surface.

Limitations. The official 128 is disjoint from the dev and held-out slices but contains the evolution selection set `items[:96]`; it is also a small slice, and its absolute numbers run a few points above the held-out 400 (77.3 versus 73.0). The held-out 400 is the more reliable estimate. `reproduce.sh` reproduces the official 128 scoreboard for every deliverable. The held-out and dev numbers need a re-run with the documented `CHARXIV_OFFSET` and `CHARXIV_NUM_SAMPLES` overrides, and the evolutionary search needs the local Qwen2.5-Coder mutator and a long GPU run. The pipeline runs on CPU, with the same greedy decoding, just much slower; a GPU is needed only to match the reported latencies. The remaining `best_accuracy` errors are mostly capability-bound for a 2B model, such as counting many ticks or dense lines, not formatting. A per-item error analysis (`cluster/error_analysis.py`, whose categorizer passes 46/46 unit tests), run on the dev slice `items[400:600]` (71 wrong of 200), finds that only 1 of those wrong items is recoverable by formatting; this is a dev diagnostic, not a held-out number, and is regenerated by re-running that script on the dev slice.

Program-genome evolution. One way to make the search itself discover the structural levers is to widen the genome from a prompt to a program. We implemented this in `evolve_program_instruct.py`. A candidate becomes a `run(ask, question, image)` program with a sandboxed greedy `ask()` primitive (32/32 self-tests, every escape class re-checked as blocked). Multi-pass candidate evaluation is about an order of magnitude more expensive than single-prompt evaluation, so within our budget the program-genome search did not pass the hand-built router. We report it as a working extension and a scaling limitation.

7 Reproducibility and conclusion

`reproduce.sh` builds the environment (`transformers>=5.0` for the `qwen3_vl` model type) and scores every deliverable on the official 128. It downloads the CharXiv images on the first run, so the submission ships without the 169 MB image set. For a fixed small VLM under an exact-match grader, the inference pipeline, not the weights, is the main lever. An LLM-driven evolutionary search can match and slightly beat careful hand-tuning on a single-prompt space, and it improves the Thinking variant too. But breaking the single-prompt ceiling needs structural ensembling: routing, gating, constrained decoding, and multi-view voting. That reaches 73.0% held-out at sub-second latency, which is 2.5 times the naive baseline.

References

[1] Wang, Z., Xia, M., He, L., Chen, H., Liu, Y., Zhu, R., Liang, K., Wu, X., Liu, H., Malladi, S., Chevalier, A., Arora, S., and Chen, D. (2024). *CharXiv: Charting Gaps in Realistic Chart Understanding in Multimodal LLMs*. Advances in Neural Information Processing Systems (NeurIPS) 37. arXiv:2406.18521.

[2] Novikov, A., Vũ, N., Eisenberger, M., Dupont, E., Huang, P.-S., Wagner, A. Z., Shirobokov, S., Kozlovskii, B., Ruiz, F. J. R., Mehrabian, A., Kumar, M. P., See, A., Chaudhuri, S., Holland, G., Davies, A., Nowozin, S., Kohli, P., and Balog, M. (2025). *AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery*. Google DeepMind, technical report. arXiv:2506.13131.

A Ablation of `best_accuracy` and extended negative results

Each lever was added in turn and measured on dev `items[400:600]`, never on the official or held-out slices. The starting point is the router with the colorbar and line gates and the multi-resolution vote.

Configuration	Dev accuracy	Δ
baseline (router + colorbar/line gates + multi-res vote)	62.0%	–
+ title-existence gate	62.5%	+0.5
+ legend-existence gate	64.5%	+2.0 **(largest)**
+ closed-class constrained decode	64.5%	+0.0
+ verbatim-decimal prompt **[default]**	64.5%	+0.0
+ crop-zoom	64.0%	−0.5 (**off by default**)

The legend-existence gate is the largest single lever, at +2.0. Constrained decoding and the verbatim-decimal clause are accuracy-neutral on dev but kept for robustness. Crop-zoom lowered accuracy and is off by default. A longer automated search probed more levers. Every one was a measured non-win.

Lever (extended search)	Measured effect	Decision
Batched inference (bs 16–32)	1.3 to 2 times raw throughput, but it breaks bit-exact greedy parity (left-padding shifts logits; 63.25 to 62.75% at bs=32). `evaluate.py` scores sequentially, so there is no graded-latency gain	Rejected; `best_speed` kept as the sequential config
Axis-label existence gate (qid2/3)	+1 question per 200 on dev, which is noise	Kept off
OCR-resolution sweep (dev)	res 2048, 1536, and 1024 give 0.645, 1280 dips to 0.635, and 768 gives 0.615; latency flat	Non-win; default 2048
Constrained decode + verbatim-decimal	neutral on dev	Kept on for robustness
Fresh 40-generation rerun (14B-Coder mutator)	winner dev 0.677, val 0.635, test-once 0.617	Confirms genome saturation; not promoted

The point of these non-wins is the breadth. The inference pipeline is near-saturated for a fixed Qwen3-VL-2B under case-insensitive exact-match greedy decoding. The verified ceiling is `best_accuracy` at 73.0% held-out.

B Evolution versus random search, and crossover

To separate the value of the quality-diversity machinery from blind perturbation, we ran the search with the LLM mutator replaced by a random-restart baseline, under matched conditions. Across four seeds per arm (15 generations each), the LLM mutator with MAP-Elites beats random restart on both the dev fitness it optimizes (mean 0.667 versus 0.609) and the once-scored held-out test winner (mean 0.560 versus 0.508). The per-seed logs are: LLM arm `abl_4819` (0.6354), `abl_4820`

(0.7500), `abl_4829` (0.6667), `abl_4830` (0.6146); random arm `abl_4821/4822/4831/4832` (0.5938/0.5938/0.6250/0.6250). A crossover ablation at `CROSSOVER_RATE=0.6` (seeds 0 and 1, logs `abl_4827/4828`, mean dev 0.615) did not beat the seed-matched mutation-only baseline on the same seeds (`abl_4819/4820`, mean dev 0.693). This is consistent with the genome saturating near 63 to 65%. Recombination cannot escape a ceiling set by the shape of the genome rather than the search operator.

C Statistics

All Wilson 95% confidence intervals, two-proportion z-tests, and the paired McNemar test are computed by `cluster/stats.py` from the logged `(correct, n)` counts in the shipped raw logs (`cluster/logs/*.out`). The output is `cluster/STATS.md`. The script derives only the inferential statistics; it does not re-estimate any accuracy. Key results: `best_accuracy` versus naive held-out gives z=12.31, p≈8e−35; versus `evolved_instruct` gives z=2.96, p=0.003 unpaired and a paired McNemar of χ^2=22.2, p=2.4e−6; `evolved` versus `manual` gives z=0.80, p=0.42, which is within noise. At n=400 the Wilson half-width near p=0.5 is $\pm$4.9 points. At n=200 (dev) it is $\pm$6.9 points, so every per-knob dev ablation delta is within noise and is reported as directional.